Assignment-1 : Regression for Electrical Engineering Problems

AI/ML for Electrical Engineering · Gati Shakti Vishwavidyalaya, Vadodara

Name	Niyati Vaishnav	**Roll no.**	24EE033
Due	26th Oct,2026	**Total marks**	100
numpy version	2.4.6	**scikit-learn version**	1.8.0

Assignment Parts:

Please write your own scripts for every regression model studied and use them on six electrical-engineering datasets. The aim is not a high accuracy score; the aim is that you can say *why* a model behaved the way it did.

Part A	Linear regression four models, and what outliers do to least squares	15
Part B	Gradient descent written from scratch (NumPy only)	20
Part C	Polynomial regression, learning curves, and extrapolation	20
Part D	Ridge, Lasso and Elastic Net on many correlated signals	20
Part E	Logistic regression for fault detection	15
Part F	Softmax regression for power-quality events	10
Part G	Caught-GenAI-being-wrong note (required, ungraded)	—

How to submit

- This sheet, filled in, as a PDF (type into it or scan your handwriting — both are fine).
- One script per part: `partA.py` … `partF.py`. Each must run from a fresh terminal with the `data/` folder next to it and print every number you wrote on this sheet.
- The figures the tasks ask for, as PNG files named as stated.
- Upload all files to Google Classroom as one zip (contains scripts, PDF and PNGs).

GenAI: You may use GenAI tools, as per the course policy. You must fill in Part G. Fill the amber *Own Predictions First* boxes before running anything; they are marked for being present and reasoned, *not* for being right. A wrong prediction you can explain is worth more than a right one you copied.

Implementation rules (follow exactly, so that answers can be checked)

R1. Use Python with NumPy, pandas, Matplotlib and scikit-learn. Part B: NumPy only (no scikit-learn).

R2. RMSE $= \sqrt{\frac{1}{m}\sum_i (\hat{y}_i - y_i)^2}$, in the units of the target.

R3. Use the split column exactly as given. Never fit anything — model, scaler, or hyperparameter — on validation or test rows.

R4. "Standardise" means `StandardScaler`, fitted on the training rows only (Part B: `np.mean`, `np.std` with `ddof=0`).

R5. Cross-validation, wherever used: `KFold(n_splits=5, shuffle=True, random_state=42)`.

R6. Leave every other scikit-learn setting at its default unless the task says otherwise.

R7. Report numbers to the decimal places shown in each table. Small differences in the last digit from library versions are acceptable.

Folder `data/`

All datasets are generated from physically motivated models with measurement noise. The true generating models will be released after the deadline so you can check your conclusions.

File	Rows	Columns (units)
`A_winding_resistance.csv`	25	`temp_C` (°C), `resistance_ohm` (Ω) — bench test of a copper winding
`A_winding_resistance_logger.csv`	28	Same test, as recorded by the plant data logger
`B_pv_plant.csv`	500	`irradiance_Wm2` (W/m^2), `module_temp_C` (°C), `ac_power_kW` (kW); no split
`C_feeder_load.csv`	380	`temp_max_C` (°C), `peak_load_MW` (MW), split: train 60 / val 300 / test_heatwave 20
`D_motor_thermal.csv`	560	30 inputs: `i_band_01`–`i_band_20` (stator-current spectral band levels, p.u., in order of frequency), `speed_rpm`, `torque_Nm`, `ambient_C`, `coolant_temp_C`, `coolant_flow_lpm`, `aux_01`–`aux_05` (spare acquisition channels, nothing connected). Target: `winding_rise_K` (K). split: train 60 / test 500
`E_bearing_condition.csv`	600	`vib_rms_mm_s` (mm/s), `kurtosis` (−), `fault` (0 healthy, 1 faulty), split: train 360 / val 120 / test 120
`F_pq_events.csv`	700	`rms_pu` (p.u.), `thd_pct` (%), `event` (normal, sag, swell, harmonic), split: train 490 / test 210

Part A — Linear Regression: Four Models [15 marks]

The DC resistance of a copper winding rises with temperature as $R(T) = R_{20}[1 + \alpha(T - 20)]$, where R_{20} is the resistance at 20°C and α is the temperature coefficient of resistance. This is linear in T: $R = \theta_0 + \theta_1 T$. Your job is to estimate R_{20} and α from a bench test (`A_winding_resistance.csv`), then see what happens to the estimate due to a data logger's glitches.

Own prediction first — fill this in BEFORE you run any code

Write θ_0 and θ_1 in terms of R_{20} and α. Roughly what value do you expect for α (look it up for copper; cite your source)?
Expanding the resistance equation gives

$$\theta_0 = R_{20}(1 - 20\alpha), \qquad \theta_1 = R_{20}\alpha$$

Hence,

$$R_{20} = \theta_0 + 20\theta_1, \qquad \alpha = \frac{\theta_1}{\theta_0 + 20\theta_1}$$

For copper at 20°C, I expect $\alpha \approx 0.00393\ ^\circ\mathrm{C}^{-1}$, based on IEEE Std 119-1974 / CRC Handbook of Chemistry and Physics.

A1. Fit $R = \theta_0 + \theta_1 T$ in four ways: (a) By hand with NumPy sums: $\theta_1 = S_{xy}/S_{xx}$, $\theta_0 = \bar{y} - \theta_1\bar{x}$. (b) Normal equation with `np.linalg.inv`. (c) `np.linalg.lstsq`. (d) `LinearRegression`. [4]

Method	θ_0 (Ω, 6 d.p.)	θ_1 (Ω/°C, 8 d.p.)
(a) S_{xy}/S_{xx} by hand	0.783905	0.00333176
(b) Normal equation (`np.linalg.inv`)	0.783905	0.00333176
(c) `np.linalg.lstsq`	0.783905	0.00333176
(d) `LinearRegression`	0.783905	0.00333176
Largest absolute difference between any two methods		1.55×10^{-15} Ω
Training RMSE of the fit (Ω, 5 d.p.)		0.00176 Ω

A2. From your fit, compute $R_{20} = \theta_0 + 20\theta_1$ and $\alpha = \theta_1/R_{20}$. [3]

R_{20} (Ω, 4 d.p.)	0.8505	α (per °C, 6 d.p.)	0.003917
Handbook α for copper (with source)	0.003930 $^\circ\mathrm{C}^{-1}$ (IEEE)	Difference from handbook (%, 1 d.p.)	−0.3%

A3. Now fit the logger file `A_winding_resistance_logger.csv` with `LinearRegression` and with `HuberRegressor` (defaults). Then list every row whose residual from the Huber fit exceeds 0.02 Ω in magnitude. [4]

Fit on logger data	R_{20} (Ω, 4 d.p.)	α (6 d.p.)	Change in α vs A2 (%)
LinearRegression (MSE)	0.8071	0.004725	+20.6%
HuberRegressor	0.8507	0.003915	−0.01%

Row Index	**Outlier Row:** `temp_C`	`resistance_ohm`	**Huber residual (Ω)**
4	35.0	1.0223	+0.1216
12	50.1	0.0000	−0.9510
19	72.4	1.1483	+0.1231

A4. Which single logger reading did the most damage to the least-squares fit, and why does squared error let it? Would you simply switch to the Huber fit in a commissioning report? What would you do instead? [4]

1. The reading at row 12, where the temperature is 50.1°C and resistance is recorded as 0 ohms , causes the greatest distortion. Least squares squares each residual, so a very large error receives disproportionately high weight and shifts the fitted parameters.
2. I would not replace the official fit with a robust model without investigation. I would inspect the logger records and test setup, identify and document the faulty measurement, exclude it only after verification, and refit the clean data using ordinary least squares. Huber regression can help identify suspicious readings.

Part B — Gradient Descent (NumPy Only) [20 marks]

NumPy only. Model: $\hat{P} = \theta_0 + \theta_1 G + \theta_2 T_m$ for the PV plant in `B_pv_plant.csv` (G = irradiance in W/m^2, T_m = module temperature in °C). There is no train/test split in this part since this is to study the optimiser, not generalisation. Write X with a leading column of ones. The gradient of the MSE is $\frac{2}{m}X^\top(X\theta - y)$ and its Hessian is $H = \frac{2}{m}X^\top X$.

Batch GD rules for this part: Start from $\theta = 0$. Stop when $(J - J^*)/J^* < 10^{-6}$, where J is the current training MSE and J^* the MSE of the closed-form solution. Give up after 200,000 iterations. Consider an execution run diverged if J exceeds 10^6 times its starting value.

Own prediction first — fill this in BEFORE you run any code

Will gradient descent converge quickly, slowly or not at all with raw units (G in hundreds, T_m in tens)? How many iterations do you guess for raw vs standardised inputs?
With raw inputs, gradient descent will converge very slowly and may not meet the iteration limit of 200,000. Irradiance has a much larger numerical scale than temperature, making the Hessian poorly conditioned. I predict that raw inputs will require over 200,000 iterations, while standardised inputs may converge in roughly 200–300 iterations because their scales are more balanced.

B1. Closed-form solution with `np.linalg.lstsq` on raw inputs. [2]

θ_0^* (kW, 4 d.p.)	θ_1^* (6 d.p.)	θ_2^* (6 d.p.)	J^* (kW^2, 6 d.p.)
0.2116	0.004929	−0.011111	0.009974

B2. Hessian and batch GD with $\eta = 1/\lambda_{\max}(H)$, on raw inputs and on standardised inputs (standardise G and T_m only with `ddof=0`, not the column of ones). [5]

Metric	**Raw inputs**	**Standardised inputs**
Eigenvalues of H (3 s.f. each)	0.0411, 183, 7.80×10^5	0.117, 2.00, 3.88
Condition number $\kappa = \lambda_{\max}/\lambda_{\min}$ (3 s.f.)	1.90×10^7	33.2
Stability limit $2/\lambda_{\max}$ (3 s.f.)	2.57×10^{-6}	0.515
Iterations to converge (or "not converged")	not converged	257
If not converged: $(J - J^*)/J^*$ at 200,000	0.0907	—

B3. Convert the standardised-input solution back to raw units and compare it with B1. Give the formulae you used and the largest absolute difference. [2]

The standardised model coefficients are converted to raw units using the mean and standard deviation of irradiance (G) and module temperature (T_m).
The conversion formulas are:

$$\theta_1 = \frac{\theta_1^{(s)}}{\sigma_G}$$

$$\theta_2 = \frac{\theta_2^{(s)}}{\sigma_T}$$

$$\theta_0 = \theta_0^{(s)} - \frac{\theta_1^{(s)}\mu_G}{\sigma_G} - \frac{\theta_2^{(s)}\mu_T}{\sigma_T}$$

The converted coefficients are $\theta_0 = 0.2110$, $\theta_1 = 0.004928$, and $\theta_2 = -0.011079$.
The largest absolute difference compared with the closed-form solution in B1 is 6.108×10^{-4}.
This small difference indicates that both approaches produce nearly equivalent solutions.

B4. Learning-rate sweep on standardised inputs: $\eta = k \times (2/\lambda_{\max})$. [4]

k	η (3 s.f.)	Status (converged / diverged)	Iterations (to converge or diverge)
0.1	0.0515	converged	1300
0.5	0.258	converged	257
0.99	0.510	converged	469
1.01	0.520	diverged	390

B5. Mini-batch gradient descent on standardised inputs. Use exactly: `rng = np.random.default_rng(0)`; at the start of every epoch `idx = rng.permutation(m)`; batches `idx[i:i+32]` in order; step counter t starts at 0 and increases by 1 per batch; step size $\eta_t = 5/(t+50)$. Report $\|\theta - \theta^*_{\text{std}}\|_2$ (distance to the standardised closed-form solution). Then try one learning schedule of your own and report it too. Save a plot of the distance against epoch as `B5_minibatch.png`. [3]

Schedule	After 1 epoch	After 5 epochs	After 20 epochs
$\eta_t = 5/(t+50)$ (4 s.f.)	0.8778	0.5844	0.3163
Yours: $\eta_t = 1/(t+20)$	1.219	0.8685	0.7485

B6. Explain, using the eigenvalues of $(I - \eta H)$, why $k = 0.99$ converged and $k = 1.01$ diverged. Then explain in one or two sentences why standardising changed the raw-input result so dramatically. [4]

1. The error in gradient descent evolves according to

   $$e_{t+1} = (I - \eta H)e_t$$

   so convergence requires every eigenvalue of $(I - \eta H)$ to have magnitude below 1. At $k = 0.99$, the largest-mode eigenvalue is approximately -0.98, so the error decreases while alternating in sign. At $k = 1.01$, it is approximately -1.02, causing the error to grow and the method to diverge.

2. Raw features have very different scales, resulting in a poorly conditioned loss surface and slow progress. Standardisation reduces this scale imbalance, improves conditioning, and allows gradient descent to converge much faster.

Part C — Polynomial Regression and Extrapolation [20 marks]

A distribution utility wants a model of daily feeder peak load against the day's maximum temperature, for summer planning (`C_feeder_load.csv`). You have 60 training days and 300 validation days, all between 8 and 40°C. A separate set of 20 days, `test_heatwave`, comes from a heatwave hotter than any training day. Do not look at the heatwave rows until task C3. Polynomial model for degree d: `make_pipeline(PolynomialFeatures(d, include_bias=False), StandardScaler(), LinearRegression())`.

Own prediction first — fill this in BEFORE you run any code

Plot the training data first. Which degree do you expect validation to choose? Will the chosen model be safe to use on a 45°C day? Why?
I expect a moderate polynomial degree, around 3 or 4, to perform well on validation data. However, the selected polynomial may not give reliable predictions at 45°C because this temperature lies outside the training range. High powers of temperature can grow rapidly beyond the observed data and produce unrealistic load estimates.

C1. Fit each degree on the training rows. Report training and validation RMSE. Degree with lowest validation RMSE: $d^* = 10$ [4]

Degree d	1	2	3	4	6	10	15
Train RMSE (MW, 1 d.p.)	80.5	30.2	27.2	25.7	24.7	24.5	24.6
Val RMSE (MW, 1 d.p.)	94.5	41.2	33.2	29.6	32.3	28.8	29.2

C2. Learning curves. Fit degree 1 and degree 10 on the first m training rows (in file order) and report RMSE on those m rows and on all 300 validation rows. Save the plot as `C2_learning_curves.png`. [4]

Subset size m	10	15	20	30	45	60
$d = 1$ train (MW, 1 d.p.)	36.0	49.8	54.1	75.9	81.0	80.5
$d = 1$ val (MW, 1 d.p.)	92.1	92.7	95.6	93.0	92.9	94.5
$d = 10$ train (MW, 1 d.p.)	13.1	12.8	22.0	23.3	23.6	24.5
$d = 10$ val (MW, 1 d.p.)	69.7	135.1	90.6	67.7	33.1	28.8

C3 & C4. C3: Evaluate on the heatwave days (training-only fits, as in C1). C4 (Physics-informed features): Heating and cooling loads switch on below and above comfort temperatures. Build four features from T: $h = \max(18-T, 0)$, $c = \max(T-24, 0)$, h^2 and c^2. Fit `make_pipeline(StandardScaler(), LinearRegression())` on them. Save one plot showing the training data, the heatwave data, your d^* model and this model over 8–47°C as `C4_extrapolation.png`. [3 + 3]

Model	Train RMSE (MW, 1 d.p.)	Val RMSE (MW, 1 d.p.)	Heatwave RMSE (MW, 1 d.p.)
C3: Degree 3	27.2	33.2	102.5
C3: Your d^* from C1 ($d = 10$)	24.5	29.0	4182.9
C3: Degree 15	23.6	393.7	1081.3
C4: Physics features (h, c, h^2, c^2)	24.7	29.8	77.6

C5. Validation picked d^*. What went wrong on the heatwave days, and what does a validation set not test? In C2, did the degree-10 training error rise as m grew — why? If you had to give the planners one model, which would it be and what warning would you attach? [6]

1. The degree-10 model achieved the lowest validation RMSE, but its heatwave RMSE increased dramatically to 10,997.3 MW. Validation data covered the same 8–40°C range as the training data, so it tested interpolation rather than performance beyond the observed range. High-degree polynomials can become unstable during extrapolation.

2. In C2, degree-10 training error increased as more samples were added because the model could no longer interpolate every point exactly. I would recommend the physics-informed model because it achieved a heatwave RMSE of 77.6 MW and represents heating and cooling behaviour more realistically. Predictions above 42°C should still be treated cautiously because real equipment and cooling loads may reach physical limits.

Part D — Ridge, Lasso and Elastic Net [20 marks]

A motor test rig logs 30 signals and the stator-winding temperature rise (`D_motor_thermal.csv`). Only 60 training runs are available; 500 more are held out for testing. Neighbouring spectral bands of the stator current are correlated with each other, and the aux channels have nothing connected. Use all 30 inputs, standardised, in every model. Settings: `RidgeCV(alphas=np.logspace(-3, 3, 61), cv=...)`; `LassoCV(cv=..., max_iter=50000, random_state=0)`; `ElasticNetCV(l1_ratio=[0.2, 0.5, 0.8], cv=..., max_iter=50000, random_state=0)`. Use the CV splitter as per Rule R5.

Own prediction first — fill this in BEFORE you run any code

How many of the 30 signals do you think really drive the temperature rise? Which ones, from your knowledge of motor losses and cooling?
I expect approximately 5–7 signals to be the main physical contributors to winding temperature rise. These include torque, fundamental and low-frequency stator-current bands, motor speed, ambient temperature, coolant temperature and coolant flow. The auxiliary channels are disconnected and should contain no genuine physical information, while higher-frequency bands are likely to have less influence on winding heating.

D1. Ordinary least squares on all 30 standardised inputs. [3]

Train RMSE / Test RMSE (K, 3 d.p.)	1.252 / 2.915
Five largest coefficients (name and value, 2 d.p.)	`torque_Nm` (+5.66), `i_band_05` (+3.26), `coolant_flow_lpm` (−3.03), `i_band_07` (+1.56), `i_band_11` (+0.98)

D2. Regularised models. Also list the features with non-zero Lasso coefficients. [4]

Model	Chosen α (3 s.f.)	l1_ratio	Test RMSE (K, 3 d.p.)	Non-zero coeffs
OLS (from D1)	—	—	2.915	30
Ridge (RidgeCV)	0.398	—	2.890	30
Lasso (LassoCV)	0.105	—	2.413	21
Elastic Net (ElasticNetCV)	0.107	0.80	2.506	23
Non-zero Lasso features: `i_band_02, i_band_05, i_band_06, i_band_07, i_band_08, i_band_09, i_band_10, i_band_11, i_band_12, i_band_13, i_band_14, i_band_15, i_band_17, i_band_18, torque_Nm, ambient_C, coolant_flow_lpm, aux_02, aux_03, aux_05` (20 features)				

D3. Selection stability: For seeds $s = 0, \ldots, 9$: `idx = np.random.default_rng(s).integers(0, 60, 60)`; fit the scaler and LassoCV (same settings) on those training rows; record which features are non-zero. Count, for each feature below, in how many of the 10 fits it was selected. Smallest and largest number of features selected in any of the 10 fits: 15 / 26 [5]

Feature	torque	band_01	band_05	band_06	band_07	band_17	flow_lpm	ambient	aux_02	aux_05
Selected (/10)	10	1	10	8	10	10	10	8	10	10

D4. The motor designer wants to know which signals drive winding temperature. Write your answer, with how confident you are. Your answer must deal with (i) `i_band_01`, which is strongly correlated with torque, (ii) the aux channels that were selected, and (iii) what cross-validation tunes α for. [8]

1. I have high confidence that torque, coolant flow and ambient temperature are important physical factors affecting winding temperature rise. However, I have lower confidence in the exact current bands selected by Lasso because neighbouring bands may be strongly correlated. For example, torque was selected in all 10 resamples, whereas `i_band_01` was selected only once.

2. Some auxiliary channels were also selected despite being disconnected. This can happen when a small training dataset contains accidental correlations. Therefore, a non-zero coefficient does not prove a physical relationship. Cross-validation chooses the regularisation parameter (α) to reduce prediction error, not necessarily to identify the true physical variables.

Part E — Logistic Regression for Bearing Fault Detection [15 marks]

A condition-monitoring system flags bearings as faulty from vibration level and kurtosis (`E_bearing_condition.c`). About 15% of bearings in the data are faulty. A missed fault means an unplanned outage; a false alarm costs an inspection. Use `LogisticRegression()` with defaults. The positive class is fault = 1.

Own prediction first — fill this in BEFORE you run any code

If a model is programmed to output always healthy, what would be the accuracy of such a model? Do you expect logistic regression to beat it by a lot?
Since approximately 51 percent of bearings are faulty, a model that always predicts "healthy" should achieve about 85 percent accuracy. Logistic regression may improve on this baseline, but the gain may be moderate because healthy samples form the majority. Its predicted fault probabilities are useful for adjusting the decision threshold to improve fault detection.

E1. Honesty floor and a one-feature model. Fit on `vib_rms_mm_s` only (raw units, not standardised). Evaluate on the test rows at threshold 0.5. [4]

Always-healthy test accuracy (%, 1 d.p.)	85.0%	Weight w and bias b (3 d.p.)	$w = 2.080$, $b = -6.835$
Decision boundary $v^* = -b/w$ (mm/s, 2 d.p.)	3.29 mm/s	Test Acc / Prec / Recall (%, 1 d.p.)	89.2% / 69.2% / 50.0%

E2. Two features (both standardised) at threshold 0.5. Fill in the test confusion matrix. [3]

True \ Predicted	Predicted Healthy (0)	Predicted Faulty (1)	Metric	Value (%, 1 d.p.)
Actually Healthy (0)	99	3	Accuracy (%)	94.2%
Actually Faulty (1)	4	14	Precision / Recall (%)	82.4% / 77.8%

E3. Moving the threshold. Maintenance wants recall of at least 80%. Using the validation rows and thresholds `np.round(np.arange(0.05, 0.951, 0.05), 2)`, pick the largest threshold τ that reaches validation recall ≥ 0.80. Then report test performance at τ. [4]

Chosen threshold τ	0.10	Highest val recall on grid (%)	83.3%
Test recall / precision at τ (%, 1 d.p.)	83.3% / 65.2%	Test false-alarm rate $FP/(FP+TN)$ (%, 1 d.p.)	7.8%
P(fault) for a bearing with $v = 2.6$ mm/s, kurtosis $= 4.2$ (3 d.p.)			0.693

E4. Why can no threshold reach 95% recall here? What would actually raise it? Which two numbers would you put in front of the maintenance manager, and why not accuracy? [4]

1. The validation recall does not reach 95 percent because the vibration and kurtosis values of some faulty bearings overlap with those of healthy bearings. Lowering the threshold cannot always separate these cases without generating many false alarms.

2. Recall could be improved by adding features such as bearing-defect frequency spectra, crest factor, peak-to-peak acceleration or high-frequency acoustic measurements. For maintenance decisions, I would report fault recall and false-alarm rate, or precision, rather than accuracy alone. With an imbalanced dataset, a model that predicts every bearing as healthy can still achieve about 85 perfect accuracy while missing all faults.

Part F — Softmax Regression for Power-Quality Events [10 marks]

A power-quality recorder labels each event as one of normal, sag, swell, harmonic from the event's RMS voltage and voltage THD (`F_pq_events.csv`). Use `make_pipeline(StandardScaler(), LogisticRegression(C=10, max_iter=2000))`; with more than two classes this is softmax (multinomial) regression.

F1. Fit on the training rows; fill in the test confusion matrix (rows: true, columns: predicted) and per-class recall. Save a scatter plot of the training data with the decision regions as `F1_regions.png`. [3]

True \ Pred	normal	sag	swell	harmonic	Recall (%, 1 d.p.)
normal	90	0	0	0	100.0%
sag	0	44	0	1	97.8%
swell	0	0	36	0	100.0%
harmonic	3	0	0	36	92.3%
Overall test accuracy (%, 1 d.p.)					98.1%

F2. Predicted class probabilities (3 d.p.). [2]

Event Operating Point	normal	sag	swell	harmonic
0.97 p.u., THD = 5.0%	0.420	0.110	0.000	0.469
0.84 p.u., THD = 8.0%	0.000	0.998	0.000	0.002

F3. How many events in the whole file are labelled sag but have THD above 6%? 19 Look at them on your plot. Is a single label the whole truth for these events? What would you build instead of softmax, and how many outputs would it have? What should an operator do with the first event in F2? [5]

1. A single event label cannot fully describe cases where voltage sag and harmonic distortion occur together. Softmax regression assigns each event to only one class, so it cannot represent simultaneous conditions.

2. A better choice is a multi-label classifier with three independent binary outputs: `is_sag`, `is_swell` and `is_harmonic`. For the first operating point (0.97 p.u., THD = 5%), I would not trip protective equipment based on the prediction alone. I would record an advisory, check whether the THD level is sustained, and inspect the harmonic filters and power-factor correction equipment.

Part G — Caught GenAI Being Wrong [Required]

Describe one place in this homework where a GenAI tool gave you something wrong, misleading, or unqualified (code, an explanation, or a number). Quote what it said, say how you found out, and what the correct version is. If it never misled you, describe the claim you checked most carefully, and how.

Where the AI was misleading: In Part B, an initial code suggestion standardised every column of the design matrix, including the bias column of ones. In Part D3, another suggestion fitted the scaler to all training rows before carrying out the resampling procedure.
How I identified the problems: The bias column has zero standard deviation, so standardising it can generate invalid values. In Part D3, fitting the scaler before resampling does not follow the instruction to fit it separately within each resample.
Correct approach: In Part B, standardise only irradiance and module temperature, then add the column of ones. In Part D3, fit a new `StandardScaler` inside each resampling iteration using only the selected training rows, and then fit the Lasso model.